Q1. Customer Satisfaction and Ordinal Encoding An e-commerce company collects customer feedback and purchase information.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import OrdinalEncoder
```

```
data=({
    "Customer_ID": (1,2,3,4,5),
    "Satisfaction": [
        "Poor", "Good", "Average", "Excellent",'Good'
    ],
    "Purchase_Amount": [
        500, 1200, 800, 2500,1200
    ],
    "Membership": [
        "Bronze", "Silver", "Platinum", "Gold",'Silver']
})

df=pd.DataFrame(data)
```

```
# define category order
satisfaction_order=[
          ["Poor", 'Average', "Good", "Excellent"]
 ]

membership_order=[
    ["Bronze", "Silver", "Gold", "Platinum"]
]
```

```
df
```

	Customer_ID	Satisfaction	Purchase_Amount	Membership
0	1	Poor	500	Bronze
1	2	Good	1200	Silver
2	3	Average	800	Platinum
3	4	Excellent	2500	Gold
4	5	Good	1200	Silver

Next steps: Generate code with df | New interactive sheet

```
# a) Apply OrdinalEncoder to Satisfaction using the order Poor < Average < Good < Excellent.
# Encode Membership using a separately defined order. Display the encoded DataFrame and verify
# programmatically that the assigned values preserve the specified rankings.

# create ordinal encoder
encoder1=OrdinalEncoder(categories=satisfaction_order)
df['Satisfaction']=encoder1.fit_transform(df[['Satisfaction']])

encoder2=OrdinalEncoder(categories=membership_order)
df['Membership']=encoder2.fit_transform(df[['Membership']])

print("Encoded DataFrame:")
print(df)

assert encoder1.categories_[0].tolist()==satisfaction_order[0]
assert encoder2.categories_[0].tolist()==membership_order[0]
```

```
print("Satisfaction ranking preserved:", True)
print("Membership ranking preserved:", True)
```

```
Encoded DataFrame:
   Customer_ID  Satisfaction  Purchase_Amount  Membership
0            1           0.0              500         0.0
1            2           2.0             1200         1.0
2            3           1.0              800         3.0
3            4           3.0             2500         2.0
4            5           2.0             1200         1.0
Satisfaction ranking preserved: True
Membership ranking preserved: True
```

```
# (b) Calculate the average purchase amount for each satisfaction category
#  before and after encoding. Explain why the numerical encoding preserves category order but doe
# equal distances between satisfaction levels or establish a causal relationship with spending.
from sklearn.preprocessing import OrdinalEncoder
befor_encoding=df.groupby('Satisfaction')['Purchase_Amount'].mean()
print(befor_encoding)
```

```
Satisfaction
0.0     500.0
1.0     800.0
2.0    1200.0
3.0    2500.0
Name: Purchase_Amount, dtype: float64
```

Q2. Label Encoding vs. Ordinal Encoding in a Classification Problem

```
data=({
   "Study_Level": [
        "Low", "Medium", "High", "Low",'High'],
    "City": [
        "Bhopal", "Delhi", "Mumbai", "Delhi",'Bhopal'],
    "Result": [
        "Fail", "Pass", "Pass", "Fail",'pass']
})
```

```
df1=pd.DataFrame(data)
```

```
# (a) Apply ordinal encoding to Study_Level using Low < Medium < High. Apply label
#  encoding to the target Result, and demonstrate why assigning integer labels to the nominal
#  feature City may introduce an artificial ordering. Display and compare the resulting represent
from sklearn.preprocessing import OrdinalEncoder,LabelEncoder
Study_order= [
        ["Low", "Medium", "High"]
]
encoder=OrdinalEncoder(categories=Study_order)
df1['Study_Level_Encoded']=encoder.fit_transform(df1[['Study_Level']])

print(df1)
```

```
  Study_Level    City Result  Study_Level_Encoded
0         Low  Bhopal   Fail                  0.0
1      Medium   Delhi   Pass                  1.0
2        High  Mumbai   Pass                  2.0
3         Low   Delhi   Fail                  0.0
4        High  Bhopal   pass                  2.0
```

```
encoder1=LabelEncoder()
df1['Result_Encoded']=encoder1.fit_transform(df1['Result'])

print(df1)
```

```
  Study_Level    City Result  Study_Level_Encoded  Result_Encoded
0         Low  Bhopal   Fail                  0.0               0
1      Medium   Delhi   Pass                  1.0               1
2        High  Mumbai   Pass                  2.0               1
3         Low   Delhi   Fail                  0.0               0
4        High  Bhopal   pass                  2.0               2
```

```
encoder_city=LabelEncoder()
df1['City_Encoded']=encoder_city.fit_transform(df1['City'])

print("Final DataFrame:")
print(df1)

for city ,value in zip(encoder_city.classes_,range(len(encoder_city.classes_))):
  print(city,'------>',value)
```

```
Final DataFrame:
  Study_Level    City Result  Study_Level_Encoded  Result_Encoded  \
0         Low  Bhopal   Fail                  0.0               0
1      Medium   Delhi   Pass                  1.0               1
2        High  Mumbai   Pass                  2.0               1
3         Low   Delhi   Fail                  0.0               0
4        High  Bhopal   pass                  2.0               2

   City_Encoded
0             0
1             1
2             2
3             1
4             0
Bhopal ------> 0
Delhi ------> 1
Mumbai ------> 2
```

```
# b) Build a preprocessing workflow that encodes the features and target appropriately, trains a
#  model, and evaluates its predictions. Explain why the encoding strategy for a feature can diff
# numerical labels for nominal features can affect certain algorithms.

from sklearn.preprocessing import OneHotEncoder
data=({
   "Study_Level": [
        "Low", "Medium", "High", "Low",'High'],
    "City": [
        "Bhopal", "Delhi", "Mumbai", "Delhi",'Bhopal'],
    "Result": [
        "Fail", "Pass", "Pass", "Fail",'pass']
})
df2=pd.DataFrame(data)
```

```
# create ordinal encoder
study_order=[
    ["Low", "Medium", "High"]
]
encoder=OrdinalEncoder(categories=study_order)
df2['Study_Encoded']=encoder.fit_transform(df2[['Study_Level']])
```

```
# create label encoder
encoder_result=LabelEncoder()
df2['Result_Encoded']=encoder_result.fit_transform(df2['Result'])
```

```
# create ohe encoder
ohe=OneHotEncoder(sparse_output=False)
ohe.fit_transform(df2[['City']])
```

```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [1., 0., 0.]])
```

```
df1
```

	Study_Level	City	Result	Study_Level_Encoded	Result_Encoded	City_Encoded
0	Low	Bhopal	Fail	0.0	0	0
1	Medium	Delhi	Pass	1.0	1	1
2	High	Mumbai	Pass	2.0	1	2
3	Low	Delhi	Fail	0.0	0	1
4	High	Bhopal	pass	2.0	2	0

Next steps: Generate code with df1 New interactive sheet

```
x=df1[
    ['Study_Level_Encoded',
    'City_Encoded']
    ]

y=df1['Result_Encoded']
```

```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)
```

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(random_state=42)
model.fit(x,y)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(random_state=42)
```

```
y_pred=model.predict(x_test)


print( y_pred)
print( y_test.values)
```

```
[1]
[1]
```

```
from sklearn.metrics import accuracy_score
accuracy=accuracy_score(y_test,y_pred)
print('Accuracy:',accuracy)
```

```
Accuracy: 1.0
```

Q3. Encoding Strategy for Mixed Categorical Data A clothing retailer stores customer preferences and order information.

```
data=({
    "Size": [
        "Small", "Medium", "Large", "Medium",
        "Small", "Large", "XL", "Medium",
        "XL", "Small", "Large", "XL"
    ],
    "Color": [
        "Red", "Blue", "Green", "Red",
        "Green", "Blue", "Red", "Green",
        "Blue", "Red", "Blue", "Green"
    ],
    "Satisfaction": [
        "Poor", "Good", "Average", "Excellent",
        "Average", "Good", "Excellent", "Poor",
        "Good", "Average", "Excellent", "Good"
    ],
    "Order_Value": [
        500, 900, 1500, 1100,
        650, 1700, 2200, 700,
        2100, 550, 1600, 2300]
})

df3=pd.DataFrame(data)
```

```
# (a) Identify which columns are ordinal and nominal. Use OrdinalEncoder with explicit category or
# . Demonstrate why Color should not be ordinal-encoded based on an invented order, and verify tha

size_order=[
    ['Small','Medium','Large','XL']
]

satisfaction_order=[
    ["Poor", "Average", "Good", "Excellent"]
]

size_encode=OrdinalEncoder(categories=size_order)
df3['Size_Encoded']=size_encode.fit_transform(df3[['Size']])

satisfaction_encode=OrdinalEncoder(categories=satisfaction_order)
df3['satisfaction_Encoded']=satisfaction_encode.fit_transform(df3[['Satisfaction']])
```

```
ohe = OneHotEncoder(sparse_output=False)
color_encoded = ohe.fit_transform(df3[['Color']])
color_df = pd.DataFrame(
    color_encoded,
    columns=ohe.get_feature_names_out(),
    index=df3.index
)

df3 = pd.concat([df3, color_df], axis=1)
```

```
print(df3[['Size', 'Size_Encoded']])
print(df3[['Satisfaction', 'satisfaction_Encoded']])
print(df3[['Color'] + list(color_df.columns)])

assert df3['Size_Encoded'].between(0, 3).all()

assert df3['satisfaction_Encoded'].between(0, 3).all()

assert df3[color_df.columns].isin([0, 1]).all().all()

print("All encodings verified successfully!")
```

```
      Size  Size_Encoded
0    Small           0.0
```

```
1   Medium           1.0
2    Large           2.0
3   Medium           1.0
4    Small           0.0
5    Large           2.0
6       XL           3.0
7   Medium           1.0
8       XL           3.0
9    Small           0.0
10   Large           2.0
11      XL           3.0
   Satisfaction  satisfaction_Encoded
0          Poor                   0.0
1          Good                   2.0
2       Average                   1.0
3     Excellent                   3.0
4       Average                   1.0
5          Good                   2.0
6     Excellent                   3.0
7          Poor                   0.0
8          Good                   2.0
9       Average                   1.0
10    Excellent                   3.0
11         Good                   2.0
    Color  Color_Blue  Color_Green  Color_Red
0     Red         0.0          0.0        1.0
1    Blue         1.0          0.0        0.0
2   Green         0.0          1.0        0.0
3     Red         0.0          0.0        1.0
4   Green         0.0          1.0        0.0
5    Blue         1.0          0.0        0.0
6     Red         0.0          0.0        1.0
7   Green         0.0          1.0        0.0
8    Blue         1.0          0.0        0.0
9     Red         0.0          0.0        1.0
10   Blue         1.0          0.0        0.0
11  Green         0.0          1.0        0.0
All encodings verified successfully!
```

```
# b) Build a ColumnTransformer that applies ordinal encoding to Size and Satisfaction and one-hot
#  to Color. Transform the dataset and explain why this combined preprocessing strategy is more a
# than assigning arbitrary integer labels to every categorical feature.
from sklearn.preprocessing import LabelEncoder,OrdinalEncoder,OneHotEncoder
from sklearn.compose import ColumnTransformer

size_order=[
    ['Small','Medium','Large','XL']
]

satisfaction_order=[
    ["Poor", "Average", "Good", "Excellent"]
]
```

```
# column transformer create
ct = ColumnTransformer(
    transformers=[
        ('size', OrdinalEncoder(categories=size_order), ['Size']),
        ('satisfaction', OrdinalEncoder(categories=satisfaction_order), ['Satisfaction']),
        ('color', OneHotEncoder(sparse_output=False), ['Color'])
    ],
    remainder='passthrough'
)


encoded_data=ct.fit_transform(df3)
```

```
df3_encoded = pd.DataFrame(
    encoded_data,
    columns=ct.get_feature_names_out(),
    index=df3.index
)
```

```
df3_encoded.head()
```

	size__Size	satisfaction__Satisfaction	color__Color_Blue	color__Color_Green	color__Color_Red
0	0.0	0.0	0.0	0.0	1.0
1	1.0	2.0	1.0	0.0	0.0
2	2.0	1.0	0.0	1.0	0.0
3	1.0	3.0	0.0	0.0	1.0
4	0.0	1.0	0.0	1.0	0.0

Next steps: Generate code with df3_encoded | New interactive sheet

```
df3_encoded.shape
```

```
(12, 11)
```

```
Start coding or generate with AI.
```